

Homework 8

Please complete the following problems. You must submit the following for each problem: a) the answer you are asked to return and b) Python code that can be run to verify your work.

1. Base Quality Distribution

Another simple quality metric for the QC is base call quality. As often quality of reads degrades over the course of a sequence, it is common practice to determine the average quality of the first, second, third,...nth base by just averaging over all reads in a file.

Figure 1 shows an example of a good quality dataset. The blue line is the median value. The yellow boxes represent the inter-quartile ranges (25-75%). The whiskers represent 10% and 90% points. As you can see the sequence reads are 40 bases long, and the median values do not fall below 37.

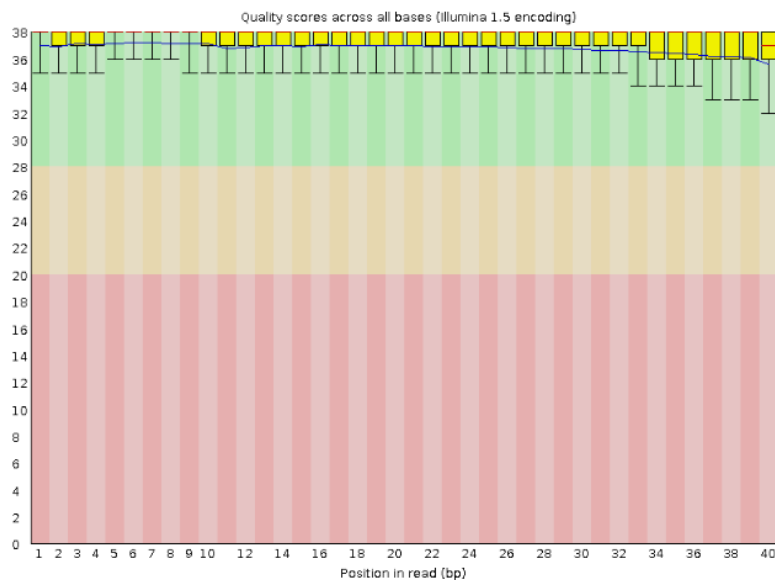


Figure 1: Per base quality scores, good dataset.

Figure 2 shows a poor dataset. The quality distribution is much wider, and average sequence quality (depicted by the blue line) is steadily declining. In some Illumina sequencing kits, the sequence quality goes up a bit first before it starts to decline, because the machine starts sequencing at lower intensity.

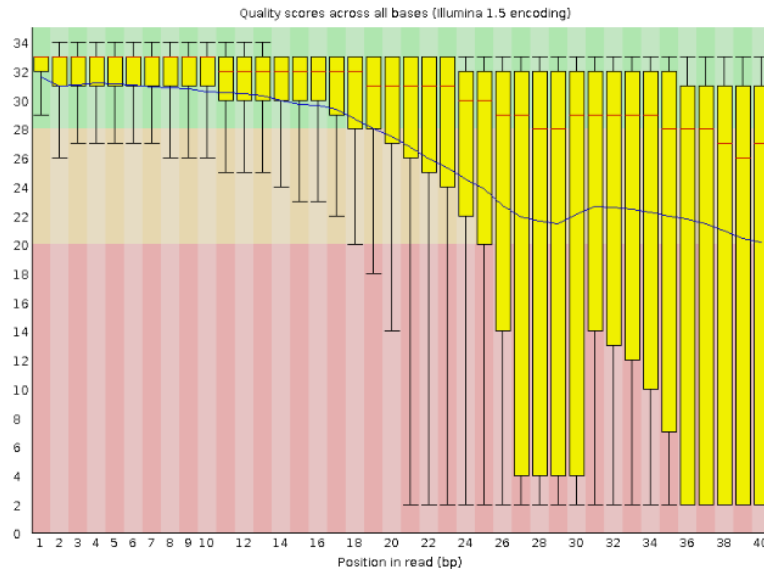


Figure 2: Per base quality scores, poor dataset.

PROBLEM

The quality of a base call depends on position in the read due to nature of the sequencing procedure. One can check this quality distribution using the "Per Base Sequence Quality" module of the FastQC program.

Average accepted quality values is 10 for the lower quartile and 25 for median. If the values fall below this limit, then the module returns a warning.

Note that FastQC will group the bases for reads >50 bp long. To show data for every base in the read use the "--nogroup" option.

Given: A FASTQ file and the quality threshold q .

Return: Number of positions where mean base quality falls below the given threshold.

Note: instead of code, please submit a screenshot of the FASTQC results.



Sample Dataset

```
26
@Rosalind_0029
GCCCCAGGGAACCCTCCGACCGAGGATCGT
+
>?F?@6<C<HF?<85486B;85:8488/2/
@Rosalind_0029
TGTGATGGCTCTCTGAATGGTTCAGGCAGT
+
@J@H@>B9:B;<D==:<;:,<::?463-,,
@Rosalind_0029
CACTCTTACTCCCTAGCCGAACTCCTTTTT
+
=88;99637@5,4664-65)/?4-2+)$)$
@Rosalind_0029
GATTATGATATCAGTTGGCTCCGAGAGCGT
+
<@BGE@8C9=B9:B<>>>7?B>7:02+33.
```

Sample Output

```
17
```

Hint: As discussed in the “PHRE: Read Quality Distribution” problem, you can use BioPython to access quality scores using the `.letter_annotations` attribute of a `SeqRecord` object returned by the `SeqIO.parse` function.



2. Base Filtration by Quality

Once you detect the bad quality letters in your reads, a good strategy is not to filter out entire reads (as done in the “FILT: Read Filtration by Quality” problem), but trim low-quality bases from both ends of your reads. It helps save a large amount of information.

PROBLEM

Bad quality bases can be easily trimmed out using certain threshold (defined by quality plot like what was done in the “BPHR: Base Quality Distribution” problem). There are a lot of trimming tools; you can try one of following:

- [FASTQ Quality Trimmer](#) tool is in Galaxy. It uses a "sliding window" approach. For a simple trimming of the ends, you should set window size 1.
- Trimmomatic is a command-line java-based tool, detail description and download link can be found [here](#). For a simple trimming from both ends, you should specify parameters LEADING and TRAILING. Trimmomatic is also in Galaxy.

Note: instead of code, please submit a screenshot of the results from your chosen trimming method.

Given: A FASTQ file and the quality cut-off value q .

Return: FASTQ file trimmed from both ends (removing leading and trailing bases with quality lower than q).

Sample Dataset

```
20
@Rosalind_0049
GCAGAGACCAGTAGATGTGTTTGC GGACGGTCGGGCTCCATGTGACACAG
+
FD@@;C<AI?4BA:=>C<G=:AE=><A??>764A8B797@A:58:527+,
@Rosalind_0049
AATGGGGGGGGGAGACAAAATACGGCTAAGGCAGGGGTCCTTGATGTCAT
+
1<<65:793967<4:92568-34:.>1;2752)24')*15;1,.3*3+*!
@Rosalind_0049
ACCCCATACGGCGAGCGTCAGCATCTGATATCCTCTTTCAATCCTAGCTA
+
B:EI>JDB5=>DA?E6B@@CA?C;=;@@C:6D:3=@49;@87;;;?8+
```



Homework

Bioinformatics Engineering

ENBC455

Sample Output

@Rosalind_0049

GCAGAGACCAGTAGATGTGTTTGC GGACGGTCGGGCTCCATGTGACAC

+

FD@@;C<AI?4BA:=>C<G=:AE=><A??>764A8B797@A:58:527

@Rosalind_0049

ATGGGGGGGGGAGACAAAATACGGCTAAGGCAGGGGTCCT

+

<<65:793967<4:92568-34:.>1;2752)24')*15;

@Rosalind_0049

ACCCCATACGGCGAGCGTCAGCATCTGATATCCTCTTTCAATCCTAGCT

+

B:EI>JDB5=>DA?E6B@@CA?C;=;@@C:6D:3=@49;@87;::;;?8



3. Introduction to Set Operations

A character is any feature (genetic, physical, etc.) that divides a collection of organisms into two separate groups. One commonly used genetic character is the possession of a single-nucleotide polymorphism, or SNP.

In a process called genotyping, the SNP markers taken from many human donors have been used very successfully to catalogue the migration and differentiation of human populations over the last 200,000 years. You can discover your own genetic heritage using one of many available commercial DNA tests.

Whether we use genetic or physical characters, we may think of a collection of n characters as a collection of "ON"/"OFF" switches. An organism is said to possess a character in the "ON" position (although often the assignment of "ON"/"OFF" is arbitrary).

A taxon is a general term used for a collection of modern-day organisms (which may be populations, subspecies, species, families, phyla, etc.) that are considered to share the same ancestor and grouped together into a single unit to construct a phylogeny. Because taxa do not generally correspond to extinct species, they are assigned to the leaves of any tree used to model a phylogeny. Given a collection of taxa, we may represent a character by the collection of taxa possessing the character.

A set is the mathematical term for a loose collection of objects, called elements. Examples of sets include $\{\textit{the moon, the sun, Wilford Brimley}\}$ and $\mathbb{R}$, the set containing all real numbers. We even have the empty set, represented by $\emptyset$ or $\{\}$, which contains no elements at all. Two sets are equal when they contain the same elements. In other words, in contrast to permutations, the ordering of the elements of a set is unimportant (e.g., $\{\textit{the moon, the sun, Wilford Brimley}\}$ is equivalent to $\{\textit{Wilford Brimley, the moon, the sun}\}$). Sets are not allowed to contain duplicate elements, so that $\{\textit{Wilford Brimley, the sun, the sun}\}$ is not a set. We have already used sets of 2 elements to represent edges from a graph.

A set A is a subset of B if every element of A is also an element of B , and we write $A \subseteq B$. For example, $\{\textit{the sun, the moon}\} \subseteq \{\textit{the sun, the moon, Wilford Brimley}\}$, and $\emptyset$ is a subset of every set (including itself!). It is a mathematical truth that a finite set with n elements has 2^n subsets. For the mathematical theory, see: http://en.wikipedia.org/wiki/Power_set.

As illustrated in the biological introduction, we can use subsets to represent the collection of taxa possessing a character. However, the number of applications is endless; for example, an event in probability can now be defined as a subset of the set containing all possible outcomes.

Just as numbers can be added, subtracted, and multiplied, we can manipulate sets in certain basic ways. The natural operations on sets are to combine their elements, to find those elements common to both sets, and to determine which elements belong to one set but not another.

Just as graph theory is the mathematical study of graphs and their properties, set theory is the mathematical study of sets and their properties.

**PROBLEM**

If A and B are sets, then their union $A \cup B$ is the set comprising any elements in either A or B ; their intersection $A \cap B$ is the set of elements in both A and B ; and their set difference $A - B$ is the set of elements in A but not in B . Furthermore, if A is a subset of another set U , then the set complement of A with respect to U is defined as the set $A^c = U - A$.

From the definitions above, one can see that $A \cup B = B \cup A$ and $A \cap B = B \cap A$ for all sets A and B , but it is not necessarily the case that $A - B = B - A$ (as seen in the Sample sections below). This set theoretical fact parallels the arithmetical fact that addition is commutative, but subtraction is not.

Given: A file containing a positive integer n ($n \leq 20,000$) and two subsets A and B of $\{1, 2, \dots, n\}$.

Return: Six sets: $A \cup B$, $A \cap B$, $A - B$, $B - A$, A^c , and B^c (where set complements are taken with respect to $\{1, 2, \dots, n\}$).

Sample Dataset

```
10
{1, 2, 3, 4, 5}
{2, 8, 5, 10}
```

Sample Output

```
{1, 2, 3, 4, 5, 8, 10}
{2, 5}
{1, 3, 4}
{8, 10}
{8, 9, 10, 6, 7}
{1, 3, 4, 6, 7, 9}
```

Hint: There are many ways to solve this problem. One idea is to write a short program for every set operation, such as a program for $A \cup B$ is shown below.

```
S = A
for b in B:
    if b not in A:
        add b to S
return S
```

The easiest solution is to take advantage of Python operators or methods for manipulating sets. Since the A and B sets in the input file are already defined in a Python-compatible manner, the `eval()` function will create sets for manipulation when reading the input file. Then, all you need to do is use the appropriate operators or methods, like these for the union of sets:



```
>>> x1 = {'foo', 'bar', 'baz'}
>>> x2 = {'baz', 'qux', 'quux'}

>>> x1.union(x2)
{'foo', 'qux', 'quux', 'baz', 'bar'}

>>> x1 | x2
{'foo', 'qux', 'quux', 'baz', 'bar'}
```


4. Genome Assembly as Shortest Superstring

Recall from the “GC: Computing GC Content” problem that almost all humans share approximately 99.9% of the same nucleotides on the genome. Thus, if we know only a few complete genomes from the species, then we already possess an important key to unlocking the species genome.

Determining an organism's complete genome (called genome sequencing) forms a central task of bioinformatics. Unfortunately, we still don't possess the microscope technology to zoom into the nucleotide level and determine the sequence of a genome's nucleotides, one at a time. However, researchers can apply chemical methods to generate and identify much smaller snippets of DNA, called reads.

After obtaining a large collection of reads from multiple copies of the same genome, the aim is to reconstruct the desired genome from these small pieces of DNA; this process is called fragment assembly (see Figure 1).

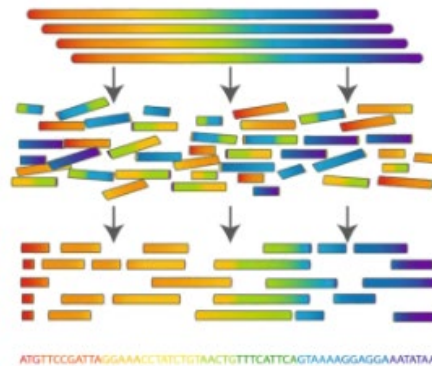


Figure 1: Fragment assembly works by blasting many copies of the same genome into smaller, identifiable reads, which are then used to computationally assemble one copy of the genome.

PROBLEM

For a collection of strings, a larger string containing every one of the smaller strings as a substring is called a superstring. By the assumption of parsimony, a shortest possible superstring over a collection of reads serves as a candidate chromosome.

Although the goal of fragment assembly is to produce an entire genome, in practice it is only possible to construct several contiguous portions of each chromosome, called contigs. A contig is a DNA string arising during genome assembly from the concatenation of overlapping reads. Contigs can vary in length, although researchers are hopeful that they will be as long as possible: the longer the contigs, the easier it is to reassemble the chromosomes making up the desired genome.

Given: A file containing at most 50 DNA strings of approximately equal length, not exceeding 1 kb, in FASTA format (which represent reads deriving from the same strand of a single linear chromosome). The dataset is guaranteed to satisfy the following condition: there exists a unique way to reconstruct the entire chromosome from these reads by gluing together pairs of reads that overlap by more than half their length.

Note that the assumption made above that reads all derive from the same strand is practically unrealistic. In reality, researchers will not know the strand of DNA from which a given read has been sequenced.



Return: A shortest superstring containing all the given strings (thus corresponding to a reconstructed chromosome).

Sample Dataset

```
>Rosalind_56
ATTAGACCTG
>Rosalind_57
CCTGCCGGAA
>Rosalind_58
AGACCTGCCG
>Rosalind_59
GCCGGAATAC
```

Sample Output

```
ATTAGACCTGCCGGAATAC
```

Hint: This is, perhaps, the most difficult Rosalind problem to be assigned at this point in the semester. The former BCE student Carina Carino wrote this guide:

1. "Initialize the list of sequences to be merged. Example: ['ATTAGACCTG', 'CCTGCCGGAA', 'AGACCTGCCG', 'GCCGGAATAC'].
2. While there is more than one sequence in the list, find the pair of sequences with the maximum overlap. (The example pair with the maximum overlap is CCTGCCGGAA and GCCGGAATAC.)
3. Merge the two sequences by removing the overlap. (The merged sequence is CCTGCCGGAATAC.)
4. Replace the original two sequences with the merged sequence. (The updated list of example sequences is ['ATTAGACCTG', 'CCTGCCGGAATAC', 'AGACCTGCCG'].)
5. Repeat steps 2-4 until there is only one sequence left in the list. (The next example pair with the maximum overlap is ATTAGACCTG and CCTGCCGGAATAC. The merged sequence is ATTAGACCTGCCGGAATAC. The updated list of sequences is ['ATTAGACCTGCCGGAATAC', 'AGACCTGCCG']. AGACCTGCCG is a substring of ATTAGACCTGCCGGAATAC. The merged sequence is ATTAGACCTGCCGGAATAC. The updated list of sequences is ['ATTAGACCTGCCGGAATAC'].)
6. Once there is only one sequence left in the list, return it as the shortest superstring. (The shortest superstring is ATTAGACCTGCCGGAATAC.)



You may use the following code to identify the overlapping sequences at the ends of the given DNA strings/reads:

```
def get_overlap_string(s1, s2):
    overlap_strings = []
    for i in range(len(s1)):
        if s1[i:] == s2[:len(s1)-i]:
            overlap_strings.append(s1[i:])
            break
    for i in range(len(s2)):
        if s2[i:] == s1[:len(s2)-i]:
            overlap_strings.append(s2[i:])
            break
    if not overlap_strings:
        return "", ""
    overlap_string = max(overlap_strings, key=len)
    return overlap_string
```

You may use the following code to create the merged sequence from two overlapping DNA strings/read:

```
def get_combined_string(s1, s2):
    combined_strings = []
    for i in range(len(s1)):
        if s1[i:] == s2[:len(s1)-i]:
            combined_strings.append(s1+s2[len(s1)-i:])
            break
    for i in range(len(s2)):
        if s2[i:] == s1[:len(s2)-i]:
            combined_strings.append(s2+s1[len(s2)-i:])
            break
    if not combined_strings:
        return "", ""
    combined_string = min(combined_strings, key=len)
    return combined_string
```

Notably, you do not have to use the provided algorithm steps nor these code examples. There are many ways to solve the problem, some of which are computationally more efficient.